2022 International Siberian Conference on Control and Communications (SIBCON)

FPGA Hardware Implementation of the Yolo Subclass Convolutional Neural Network Model in Computer Vision Systems

Nikolay G. Markov
Dept.Information Tecnologies
Tomsk Polytechnic University
Tomsk, Russian Federation
markovng@tpu.ru

Ivan V. Zoev
JSC «Infotecs»
Tomsk, Russian Federation
ivz3@tpu.ru

Evgeniy A. Mytsko
Dept.Information Tecnologies
Tomsk Polytechnic University
Tomsk, Russian Federation
evgenvt@tpu.ru

2022 International Siberian Conference on Control and Communications (SIBCON) | 978-1-6654-7628-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/SIBCON56144.2022.10003015

***Abstract* — The computing unit (CU) of the computer vision system (CVS) has been developed based on a system on a chip (SOC) with the Xilinx Field-Programmable Gate Array (FPGA). A new model of a convolutional neural network (CNN) tiny-YOLO-Inception-ResNet2 related to neural networks of the YOLO subclass was proposed. A feature of this model is the presence of two Inception-ResNet modules. The weight coefficients of the trained software-implemented CNN of the same model were used. Images from the Pascal VOC 2007 dataset were used to train and test these CNN models. The research of the CU effectiveness was carried out. Firstly, we study the detection time of one image which spent in each layer of the CNN model and of the whole model depending on the number of universal computing units in the CU. Also, the study of the detecting objects accuracy depending on the bit depth (16 or 32 bits) of floating-point numbers was carried out. It is concluded that it is necessary to perform calculations using 32-bit floating-point numbers. It is shown that the power consumption of the CU did not exceed 12 Watts during all experiments.**

Keyword — field programmable gate array, computing unit, computer vision system, hardware implementation, convolutional neural network, YOLO subclass.

I. INTRODUCTION

Nowadays in Russia and in many foreign industrialized countries there is a problem of monitoring and detecting illegal actions at hazardous industrial objects and the problem of timely detection of fires at such objects and in the territories arround. The solution of these problems, first of all, is relevant for oil and gas enterprises that have dangerous technological facilities in the fields during the extraction and preparation of hydrocarbon raw materials and extended (hundreds and thousands of kilometers) main oil and gas pipelines for transporting prepared raw materials. The spacecraft and unmanned aerial vehicles (UAVs) with a computer vision system (CVS) implemented on board are increasingly being are used to solve such problems. It make possible to monitor technological objects and adjacent territories [1¬]. Such CVS can be based on hardware-implemented convolutional neural networks (CNNs). The use of hardware-implemented CNN on programmable logic integrated circuits (FPGA) of modern systems on a chip (SoC) will allow real-time analysis of images obtained by monitoring the earth's surface. However, when developing such SoC, it is necessary to find a balance between the speed of the SoC computing device created on modern SoC and providing real-time image analysis with the high accuracy of recognizing objects on the earth's surface. In addition, a balance should be sought between the mass and power consumption of this device in order to increase the flight time of the UAV.

The analysis shows that the YOLO subclass [2 ¬ 7] are the most suitable CNN models for solving the problem of detecting objects in images. These CNN models provide higher accuracy and speed of object detection in images than CNN models of other subclasses.

The paper solves the problem of hardware implementation of a new YOLO CNN model on the FPGA as part of a computing unit (CU) of the CVS and presents the results of the device effectiveness study in recognizing objects in images.

II. ANALYSYS OF THE KNOWN CNN MODELS OF THE YOLO SUBCLASS

Let's analyze the most famous models of neural networks of the YOLO subclass. The YOLOv2 CNN model is considered to be well studied and promising [2, 3, 6]. It has 22 convolutional layers and the leaky ReLU function is used as an activation function defined by the formula max (0.1x, x). This model has a number of differences from the original YOLO CNN model. Firstly, the YOLOv2 model is a fully convolutional network since two fully connected layers at the output are removed from its architecture which makes it possible not to use the dropout method at the stage of its training. Second, to detect features of different scales, the feature maps before the last subsample layer are combined with the output feature maps of the output layer. Offsets are applied relative to the anchor values (anchor boxes) to determine the bounding areas (usually rectangles) of objects in images. It is calculated using the training set. Such calculations are carried out using the K-means method. The accuracy of object detection using the YOLOv2 CNN model according to the mAP (mean Average Precision – mAP) metric is 76.8% on the Pascal VOC 2007 sample [8]. The execution speed of this model reaches 67 frames per second at this accuracy on the NVIDIA Titan X graphics card.

Due to limited computing resources in most CVSs a simpler tiny-YOLO model with only 9 convolutional layers (15 layers in total) is promising for hardware implementation [2, 3]. This architecture does not use the addition of feature maps from previous layers to subsequent ones. However, the detection accuracy and network execution speed are quite close to the corresponding values in the case of using the YOLOv2 architecture. The architecture of such a CNN is shown in Table 1.

Note that in all YOLO subclass models the batch-normalization operation is used after each convolutional

Authorized licensed use limited to: National Taiwan Univ of Science and Technology. Downloaded on October 23,2023 at 16:15:36 UTC from IEEE Xplore. Restrictions apply.

layer. This complicates the process of calculating such a model. Additional computing resources should be used to repeatedly perform such an operation with the hardware implementation and the power consumption of the CU will increase. In this regard it is necessary to develop new CNN models without batch normalization.

TABLE I. TINY-YOLO CNN MODEL

№	Layer type	Cores count	Core size/step	Layer input dimensions
1	Convolutional	16	3x3/1	416x416x3
2	Subsampling	-	2x2/2	416x416x16
3	Convolutional	32	3x3/1	208x208x16
4	Subsampling	-	2x2/2	208x208x32
5	Convolutional	64	3x3/1	104x104x32
6	Subsampling	-	2x2/2	104x104x64
7	Convolutional	128	3x3/1	52x52x64
8	Subsampling	-	2x2/2	52x52x128
9	Convolutional	256	3x3/1	26x26x128
10	Subsampling	-	2x2/2	26x26x256
11	Convolutional	512	3x3/1	13x13x256
12	Subsampling	-	2x2/1	13x13x512
13	Convolutional	1024	3x3/1	13x13x512
14	Convolutional	1024	3x3/1	13x13x1024
15	Convolutional	125	1x1/1	13x13x1024
16	Detector	-	-	

III. NEW CNN MODEL OF THE YOLO SUBCALSS

Today various methods and methods can reduce the cost of computing in the CNNs and the computing resources used while maintaining the accuracy of the neural network in comparison with deeper versions of neural networks. For example, on the imagenet sample the approach using inception modules which were used in the googlenet CNN [9] has proven itself well.

The article [10] presents the results of a study that showed that an increase in the number of CNN layers does not lead to an increase in the detection accuracy. Along with this, a new architecture was proposed using ResNet modules, where the data at the input of the layer is also fed to the input of the next one through two layers. This makes it possible to use a smaller number of the CNN layers model with a higher accuracy of object detection.

A combination of modules is possible when a ResNet module is embedded in the Inception module. A detailed diagram of the Inception-ResNet module is presented in the article [1]. As a result the amount of calculations is reduced in comparison with deeper neural networks but the accuracy of object detection is maintained. Using this approach to develop the tiny-YOLO CNN architecture and replacing some of the model layers with Inception-ResNet modules in various combinations you can get a new tiny-YOLO-Inception-ResNet model. We can get the tiny-YOLO-Inception-ResNet2 and tiny-YOLO-Inception-ResNet3 models using two and three Inception-ResNet modules in series. Studies have shown that among these three new models the tiny-YOLO-Inception model is the most effective is tiny-YOLO-Inception-ResNet2. An important difference of this proposed model from the original tiny-YOLO CNN is the refusal to use the batch normalization operation. Table 2 shows the tiny-YOLO-Inception-ResNet2 CNN model.

TABLE II. TINY-YOLO-INCEPTION-RESNET2 CNN MODEL

№	Layer type	Cores count	Core size/step	Layer input dimensions
1	Convolutional	16	3x3/1	416x416x3
2	Subsampling	-	2x2/2	416x416x16
3	Convolutional	32	3x3/1	208x208x16
4	Subsampling	-	2x2/2	208x208x32
5	Convolutional	64	3x3/1	104x104x32
6	Subsampling	-	2x2/2	104x104x64
7	Convolutional	256	3x3/1	52x52x64
8	Subsampling	-	2x2/2	52x52x256
9	Inception ResNet	-	-	26x26x256
10	Inception ResNet	-	-	26x26x128
11	Subsampling	-	2x2/2	26x26x128
12	Convolutional	256	3x3/1	13x13x128
13	Subsampling	-	2x2/1	13x13x256
14	Convolutional	128	3x3/1	13x13x256
15	Convolutional	125	1x1/1	13x13x128
16	Detector	-	-	

The Inception ResNet layer which dualization is presented in Table 3 includes a set of 7 convolutional layers and an adder (layer 8). The order of the layers is indicated in the columns «Prev.» (previous layer number) and «Next.» (Next layer number).

TABLE III. INCEPTION-RESNET2 LAYER DETALIZATION

№	Cores count	Core size/step	Layer input dimensions	Prev.	Next
1	32	1x1/1	26x26x256	-	3
2	32	1x1/1	26x26x256	-	4
3	32	3x3/1	26x26x32	1	7
4	32	3x3/1	26x26x32	2	6
5	32	1x1/1	26x26x256	-	7
6	32	3x3/1	26x26x256	4	7
7	256	1x1/1	26x26x96	3	8
8	-	-	26x26x256	7	-

According to Table 3, layers 1 and 2, 3 and 4, 5 and 6 work in parallel. The output data of layers 3, 5, 6 are combined by concatenation in layer 7. The results of layer 7 are fed to the adder (layer 8).

IV. DEVELOPMENT AND RESEARCH OF A COMPUTING DEVICE

The CU was developed on the Xilinx Zynq 7000 SoC (Kintex FPGA) with a hardware-implemented CNN tiny-

Authorized licensed use limited to: National Taiwan Univ of Science and Technology. Downloaded on October 23,2023 at 16:15:36 UTC from IEEE Xplore. Restrictions apply.

YOLO-Inception-ResNet2 model. In the hardware implementation of such a CNN model, an original method for organizing calculations in a hardware CNN on an FPGA was used, proposed in [1] and differing from the known methods by using unified universal computing units (UCU) for convolution and subsampling procedures. The universality of blocks lies in the ability to perform both convolution and subselection operations with the same block. In turn, the set of NCUs, the number of which can be configured depending on the available FPGA resources, forms a neurocomputing unit (NCU) [1].

As weight coefficients for the hardware-implemented CNN, the weight coefficients of the trained software-implemented CNN of the same model were used. The number of UCUs in the NCU was set from 32 to 128 (limited by FPGA resources). The clock frequency of the device is set to 100 MHz. As training and test samples, images from the well-known Pascal VOC 2007 dataset were used in which the objects in the images belong to 20 classes. The test sample consisted of 4952 images.

TABLE IV. SINGLE IMAGE DETECTION TIME WITH TINY-YOLO-INCEPTION-RESNET2 MODEL

Layer type	№	Single image detection time, ms		
		32 UCU	*64 UCU*	*128 UCU*
Convolutional	1	126,375	125,755	125,669
Subsampling	2	79,671	79,157	79,008
Convolutional	3	123,031	74,416	74,363
Subsampling	4	39,697	36,376	36,261
Convolutional	5	102,198	58,4	39,083
Subsampling	6	23,067	20,034	20,425
Convolutional	7	757,277	100,857	58.895
Subsampling	8	29,785	20,644	22,743
Inception-ResNet	9	25,202	14,58	10,39
	10	3,569	2,481	2,156
	11	3,599	2,476	2,135
	12	25,628	14,845	10,111
	13	3,395	2,474	2,144
	14	24,142	13,835	10,332
	15	74,718	56,762	54,923
Sum	16	4,324	4,358	4,312
Convolutional	17	147,058	65,881	35,233
Inception-ResNet	18	25,224	14,532	10,337
	19	3,572	2,434	2,148
	20	3,604	2,498	2,164
	21	25,599	14,883	10,154
	22	3,378	2,449	2,198
	23	24,131	13,856	10,334
	24	74,725	56,758	54,967
Sum	25	4,32	4,339	4,318
Convolutional	26	147,015	65,842	35,25
Subsampling	27	2,822	2,929	2,678
Convolutional	28	178,596	70,476	11,912
Subsampling	29	3,461	3,006	3,016
Convolutional	30	42,697	21,459	13,303
Convolutional	31	87,611	35,641	7,215
Total		2219,491	1004,433	699,282

Consider the results of the CU study. Table 4 shows the detection time of one image spent on calculations in each layer of the tiny-YOLO-Inception-ResNet2 CNN model in the presence of 32, 64 and 128 universal computing units in the NCU.

The values of the detection time in Table 4 are obtained as the average time for performing calculations in layers and in the hardware model of the CNN as a whole for all images of the test sample. The results show that an increase in the number of computational units in an NCU leads to an almost linear increase in its performance. The main contribution to performance belongs to the original parallel computing algorithm used in the layers of the CNN model and the kernel caching algorithm. It is worth noting that the decrease in the detection time in each layer of the model with an increase in the number of computational blocks in the NCU practically does not occur for the initial layers of the CNN (layers 1, 2 and 4). The reason for this is the peculiarities of the algorithms used when working with large volumes of output data prepared for uploading to external memory in the first layers of the tiny-YOLO-Inseption-ResNet2 CNN model.

Note that the power consumption of the CU during the research did not exceed 12 W.

Also, studies were carried out on the accuracy of detecting objects in images for the tiny-YOLO-Inseption-ResNet2 CNN model, depending on the bit depth (16 or 32 bits) of floating-point numbers used in the CU. Images from the PascalVOC2007 dataset were also used as images for testing. It turned out that in the case of using 32-bit floating-point numbers in the CU the results in terms of detection accuracy are close (taking into account the computational error of the CU) to the results obtained in the study of the detection accuracy of the software implementation of the CNN tiny-YOLO-Inseption-ResNet2 model. For an example that clearly shows the drop in detection accuracy when using a small floating-point number. The results of detection using this hardware-implemented CNN of the same image from this dataset are given. Figure 1 shows the result of detection carried out using this software-implemented CNN model, and in figure 2 - with the help of a hardware-implemented CNN model.

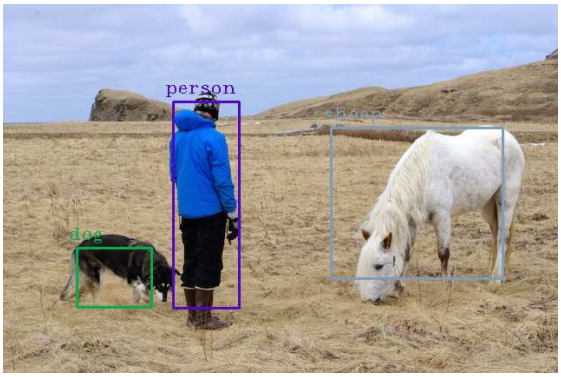

Fig. 1. Results of three objects detection (software implementation of the CNN model, 32-bit floating-point numbers).

Authorized licensed use limited to: National Taiwan Univ of Science and Technology. Downloaded on October 23,2023 at 16:15:36 UTC from IEEE Xplore. Restrictions apply.

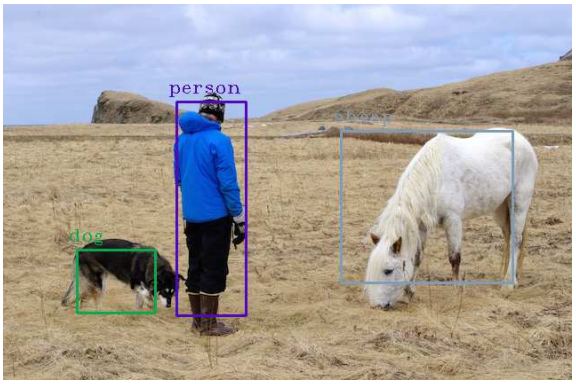

Fig. 2. Results of three objects detection (hardware implementation of the CNN model, 32-bit floating-point numbers).

Calculations over 32-bit floating-point numbers were used. In Figures 1 and 2, it is difficult to visually identify the difference in the contours of the detected objects (it is insignificant, but still exists).

Figure 3 shows the results of detecting objects on the same image in the case of calculations over 16-bit numbers. It can be seen that during detection one of the objects was lost. Such an example shows the need to use 32-bit floating-point numbers in the CU especially if the device will be used in CVS when solving critical tasks of detecting objects in images.

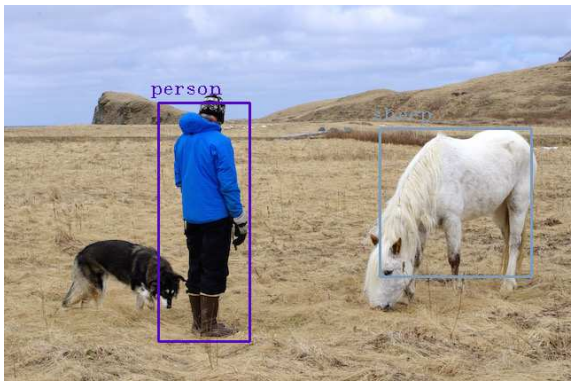

Fig. 3. Results of three objects detection (hardware implementation of the CNN model, 16-bit floating-point numbers).

V. Conclusion

It is shown that among the most suitable CNN models for solving the problem of detecting objects in images in the CVS today preference should be given to neural networks of the YOLO subclass. An analysis of the known YOLO subclass CNN models showed that in order to increase the computational speed while maintaining the accuracy of image detection in such models it is necessary to replace some of the layers with Inception-ResNet modules in various combinations. A new CNN model tiny-YOLO-Inseption-ResNet2 was obtained by replacing a number of layers in the original tiny-YOLO model with a sequence of two Inception-ResNet modules. It also abandoned the use of batch normalization.

A CU was developed on the Xilinx Zynq 7000 SoC (Kintex FPGA) with the tiny-YOLO-Inception-ResNet2 CNN model implemented on the FPGA. As its weight coefficients in the hardware implementation the weight coefficients of the trained software-implemented CNN of the same model were used. Images from the Pascal VOC 2007 dataset were used to train and test these CNN models.

The CU effectiveness studies have been carried out. Firstly, the detection time of one image spent on calculations in each layer of the CNN model and in the model as a whole, depending on the number of universal computing units in the CU, was studied. It is shown that an increase in the number of such blocks in the CU leads to an almost linear increase in its performance. The accuracy of detecting objects in images was also studied depending on the bit depth (16 or 32 bits) of floating-point numbers used in the CU. It was found that in order to maintain the accuracy of detecting objects in images in the CVS it is necessary to carry out calculations in the CU over 32-bit floating-point numbers. It is also shown that the power consumption of the CU during all experiments did not exceed 12 W. The results of the research indicate the prospects for using such a CU as part of the CVS installed on the UAV.

References

[1] I.V. Zoev, N.G. Markov, S.E. Ryzova. “Intelligent computer vision system for unmanned aerial vehicles for monitoring technological objects of oil and gas industry”. Bulletin of the Tomsk Polytechnic University. Geo Assets Engineering; vol. 330, pp. 34–49, 2019.

[2] Y. J. Wai, Z. M. Yussof, S.I. Salim. A scalable FPGA based accelerator for Tiny-YOLO-v2 using OpenCL. International Journal of Reconfigurable and Embedded Systems (IJRES)., vol 8, pp. 206 - 214, 2019.

[3] J. Redmon, A. Farhadi. YOLO9000: better, faster, stronger. IEEE Conference on Computer Vision and Pattern Recognition, CVPR, Honolulu, HI, USA, pp. 6517–6525, July 21-26, 2017.

[4] D.T. Nguyen, T.N. Nguyen, H. Kim, H-J Lee. A High-Throughput and Power-Efficient FPGA Implementa-tion of YOLO CNN for Object Detection. IEEE Transactions on Very Large Scale Integration (VLSI) Systems; vol. 27, pp. 1861–1873, 2019.

[5] S. Zhang, J. Cao, Q. Zhang, Q. Zhang, Y. Zhang, Y. Wang. An FPGA-Based Reconfigurable CNN Accelerator for YOLO. IEEE 3rd International Conference on Electronics Technology (ICET), pp. 74-78, 2020.

[6] Z. Zhichao, M.A. MAHMUD, A.Z. Kouzani. Re-source-constrained FPGA implementation of YOLOv2. Neural Computing and Applications, vol. 34, pp. 16989–17006, 2022.

[7] I.V. Zoev, A.P. Beresnev, N.G. Markov. Convolutional neural networks of the YOLO class in computer vision systems for mobile robotic complexes. Control and Communications Proceedings of International Siberian Conference, Russia, Tomsk, pp. 1-5, 2019.

[8] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2007 (VOC2007) Results. http://www.pascalnetwork.org/challenges/VOC/voc2007/workshop/index.html.

[9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich. Going Deeper with Convolutions. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1-9, 2015.

[10] C. Szegedy, S. Ioffe, V. Vanhoucke, A.A. Alemi. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. Proceedings of the Thirty-First AAAI Conference on Artificial IntelligenceFebruary, pp. 4278–4284, 2017.

Authorized licensed use limited to: National Taiwan Univ of Science and Technology. Downloaded on October 23,2023 at 16:15:36 UTC from IEEE Xplore. Restrictions apply.